

Threads I

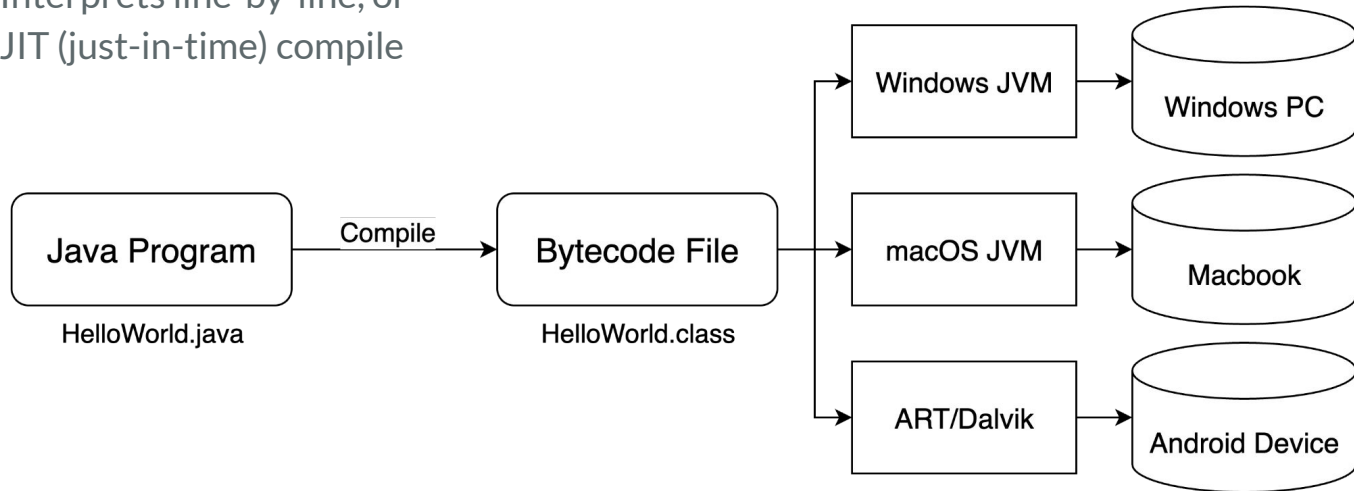
Java Full Stack Program

Outline

- JVM, JRE, JDK
- Thread VS. Process
- Thread in Java
- Thread Lifecycle
 - main thread
 - Lock, synchronization
 - Daemon Threads
- Thread Safety
- Synchronized keyword
- Singleton Design Pattern

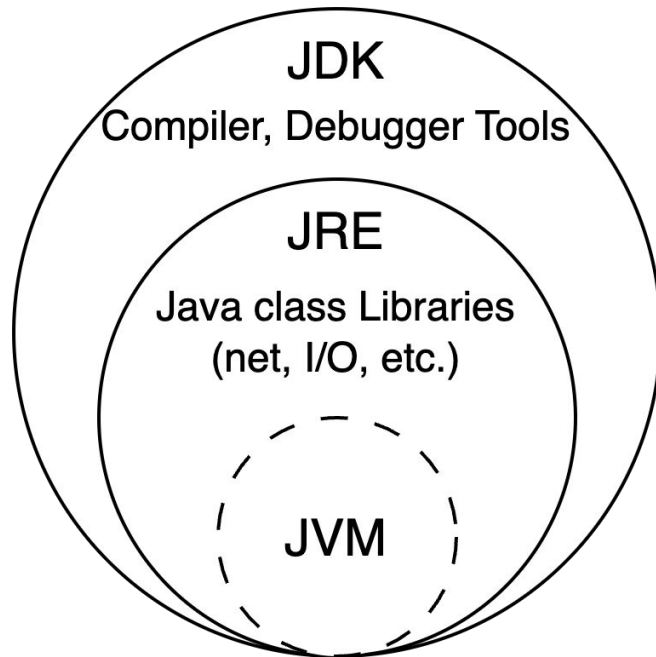
How a Java Program is Executed

- Write Java source code (.java file)
- Java Compiler **compiles** the source code into bytecode (.class file)
- The bytecode is platform-independent and can run on any JVM
- The JVM translates the bytecode into native machine code for the host system
 - Interprets line-by-line, or
 - JIT (just-in-time) compile



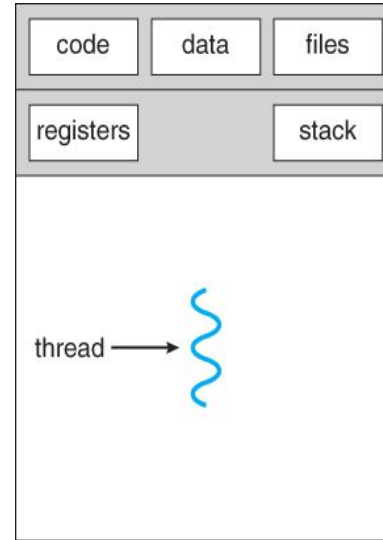
JVM, JRE, JDK

- Java Virtual Machine (JVM)
 - Abstract concept
 - Platform-independent
 - Computing machine
 - Run **bytecode** files
 - Java Runtime Environment (JRE)
 - Implementation of JVM
 - Libraries + other files
 - Java Development Kit (JDK)
 - Toolkit for Java developers
 - JRE + Development tools
 - Compiler, debugger, etc.
-
- JVM handles the processes and threads

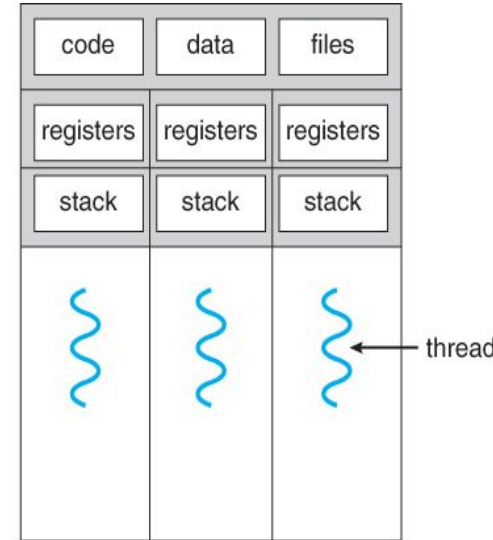


Process VS. Thread

- A process is a program in execution
 - Self-Contained Execution Environment
 - Everything needed to run independently
 - Private memory space: Each process has its own
 - Isolated: One process won't affect others if it fails.
- A thread is the smallest unit of processing, a *lightweight* sub-process
 - One process has ≥ 1 thread(s)
 - Each thread has a separate path of execution
 - If exception occurs in one thread, it doesn't affect other threads.
 - Threads in the same process share the code, data (static variables, heap data, files), making communication easier but requiring careful synchronization.



single-threaded process



multithreaded process

Threads in Java

- In plain Java, there are two components working closely with threads:
 - A class - `java.lang.Thread`
 - An interface - `java.lang.Runnable`
- 2 approaches to create a new thread
 - Extend the **Thread** class
 - Implement the **Runnable** interface

java.lang.Thread

- In Java, there is a Thread class, which belongs to the java.lang package, declared as

```
public class Thread
    extends Object
    implements Runnable
```

- This class encapsulates any thread of execution. Any threads that are created are the instances of this class, which contain the run() method in it. The functionality of the thread (the *task*) can be achieved by overriding this run() method.

```
@Override
public void run(){
    // the task we want to do
}
```

java.lang.Thread - Example

- We have a print number 1 to 10
- Extend the thread
- Create an printer instance
 - start() the printer
 - run() the printer
 - ...

```
class Printer extends Thread {
    @Override
    public void run() {
        for (int i = 1; i <= 10; i++) {
            System.out.println(i);
        }
    }
}

public class PrintNumber {
    public static void main(String[] args) {
        Printer p = new Printer();
        p.start();
        //...
        p.run();
        p.run();
    }
}
```

java.lang.Runnable

- `run()` – “A task that can be run”
 - Threads execute code through their `run()` method, but sometimes we need to separate the task from the thread itself
 - `Runnable` allows us to define tasks independently - they can be created once, passed around, executed by different threads, or even stored for later use
- `java.lang.Runnable` is a functional interface
 - With single `run()` method
 - Calling the `run()` means executing the task
 - Starting from Java 8, we can use lambda functions to implement it directly

```
@FunctionalInterface
public interface Runnable {
    public void run();
}
```

java.lang.Runnable - Example

- Create a Runnable

```
class PrintNumberTask implements Runnable {  
    @Override  
    public void run(){  
        for(int i = 0; i < 10; i++) {  
            System.out.println(i);  
        }  
    }  
}
```

```
Runnable printNumberTask = () -> {  
    for(int i = 0; i < 10; i++) {  
        System.out.println(i);  
    }  
}
```

- Start a thread and pass the Runnable in

```
Thread t = new Thread(printNumberTask);  
t.start();
```

-

Runnable VS. Thread

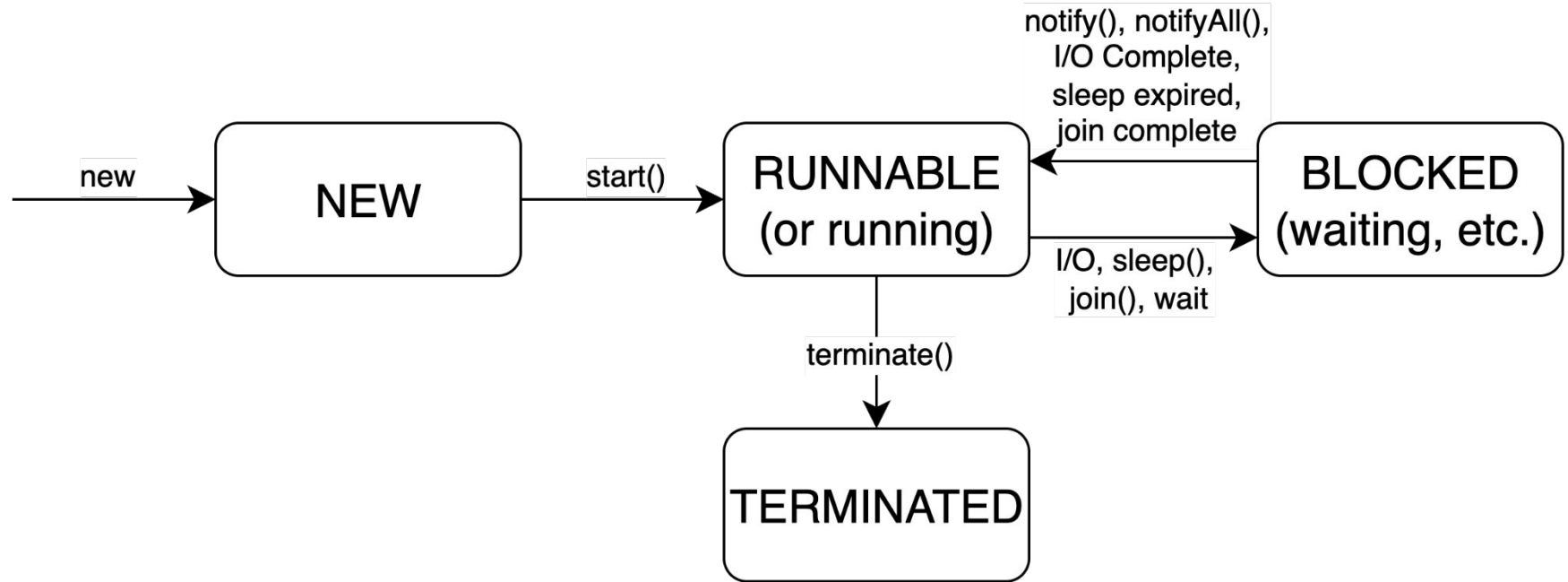
- Difference
 - Extending a class vs. implement an interface
 - Flexibility, code reuse, ...
 - Resource sharing: multiple threads can share the same Runnable instances
 - For example, selling tickets from multiple windows
 - Java 8 allows us to use lambda expression to directly implement Runnable
- Simply put, we generally recommend the use of Runnable over Thread.

Thread Lifecycle

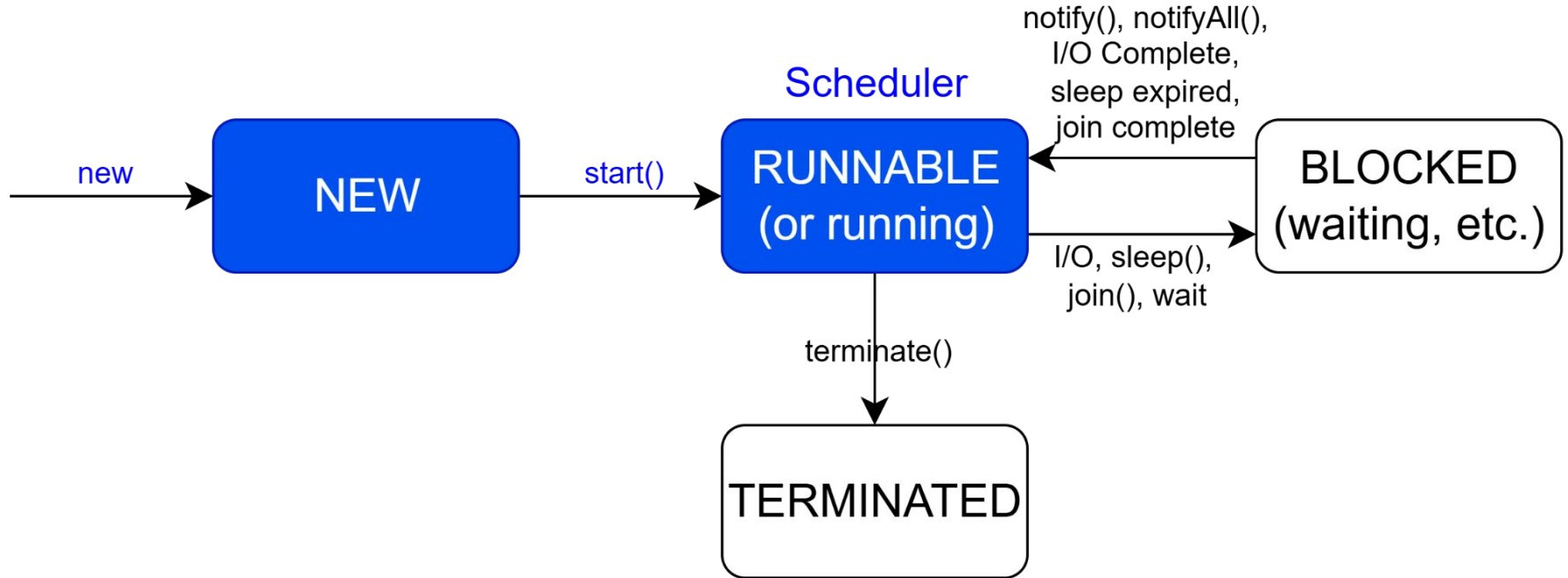
States of thread — managed by *Thread Scheduler* of JVM

- NEW — just created but not started
- RUNNABLE — created, started, and able to run
 - running — thread is currently running
- BLOCKED, WAITING, TIMED_WAITING — created and started but unable to run because it is waiting for some event to occur
 - BLOCKED: waiting to acquire a lock (e.g., synchronized)
 - WAITING: waits indefinitely (e.g., join(), wait())
 - TIMED_WAITING: waits for a specified time (e.g., sleep(), wait(ms), join(ms))
- TERMINATED — thread has finished or is stopped

Thread Lifecycle



Thread Lifecycle



start(), run()

```
Thread t = new Thread(printNumberTask);  
t.start();
```

- When a program calls the start() method, an actual thread is created and then the run() method is **executed once in that new thread**
 - Create an object t of Thread
 - Call t.start()
- When a program calls the run() method, no new thread will be created
 - run() method will be executed as a normal method call on **the current calling thread** itself
 - Multithreading does not necessarily take place
- Code Example
 - Can we call the start() method on a thread twice?
 - Can we call the run() method on a thread twice?

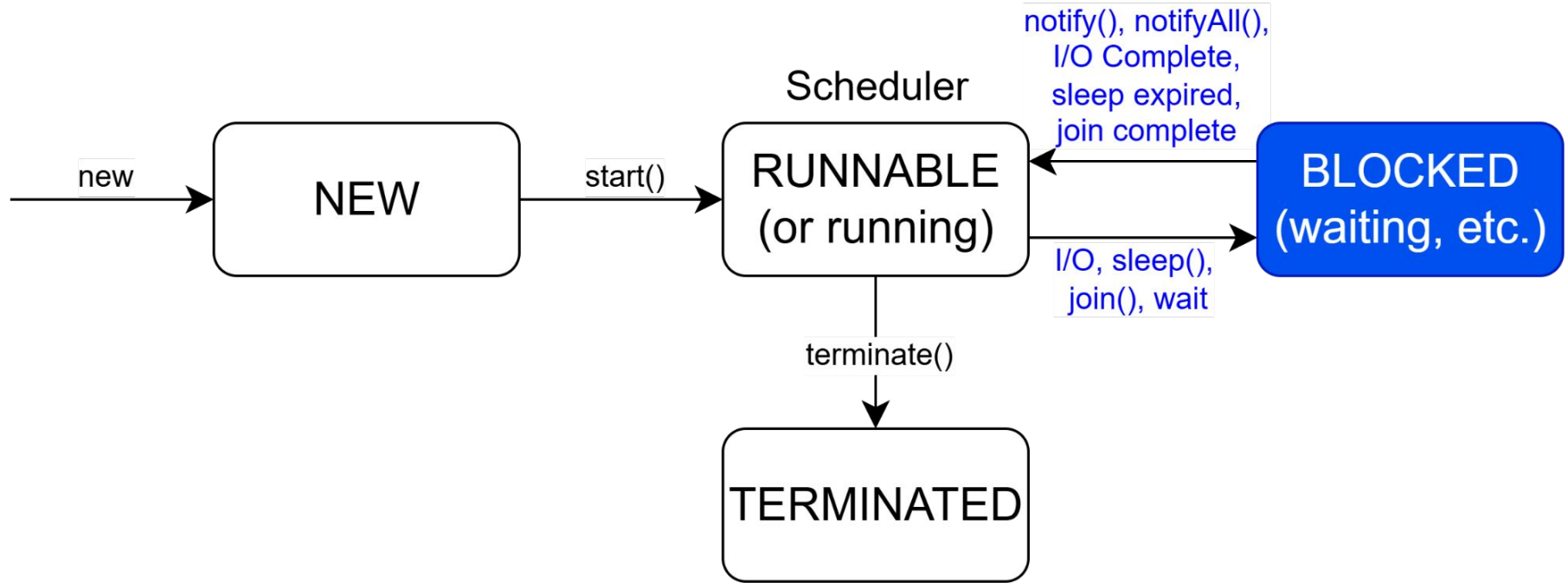
Interview Question: start() VS. run()

- Behavior
- Multiple invocation?
- Definition?
 - start() is defined in java.lang.Thread class
 - run() is defined in java.lang.Runnable interface. Must be overridden ...

Main Thread

- `main()` method – entry point of every java program
- main thread – default thread of every java program
- When the execution of Java program starts, the JVM creates the main thread and calls the program's `main()` method within that thread.
 - Programmers can always take control of the main thread
 - Apart from this, JVM also creates some invisible threads, which are important for JVM housekeeping tasks

Thread Lifecycle



Blocking States and Related Methods

- `Thread.sleep()`
 - causes the current thread to suspend execution for a specified period
 - `Thread.sleep(x)` makes the thread sleep for x **milliseconds**
- `Thread.join()`
 - The method allows one thread to wait for the completion of another
 - For example:
 - we create `Thread t = new Thread();` in the main thread.
 - Calling `t.join()` will cause the main thread to wait until Thread t finishes execution.
 - `join(timeout)`: This means the main thread will wait for the new thread to finish for up to timeout ms. Afterwards, if the new thread is still running, the main thread will resume

Lock in Java

- Example: Producer and Consumer
 - Buffer: shared storage to store data
 - Producer: puts data into buffer
 - When buffer is full → wait for consumers
 - Consumer: gets data out of buffer
 - When buffer is empty → wait for producers
- Threads need to use resources. Each resource has a lock. Only the thread holding the lock can access the resource
 - In this example, threads = Producer and Consumer; resource = buffer
- Synchronization in Java is a mechanism to control access to shared resources when multiple threads are running concurrently

Lock in Java - Related Methods

- Object.wait()
 - Causes the current thread to wait until another thread invokes notify() or notifyAll() on the same object
 - Must be called inside a synchronized context, or it will throw IllegalMonitorStateException
 - When a thread calls wait(), it **releases the lock** on the object and enters the WAITING state
- Object.notify()
 - Wakes up one thread that is waiting on the object's monitor
 - The awakened thread will proceed after it re-acquires the object's lock
 - Must be called within a synchronized context
- Object.notifyAll()
 - Wakes up all threads that are waiting on the object's monitor
 - Must be used within a synchronized block or method

Interrupt

- `Thread.interrupt()`
 - A method used to signal a thread to stop what it's doing.
 - It does not forcibly stop the thread — it simply sets an interrupt flag.
 - If the thread is in a blocking state (e.g., `sleep()`, `wait()`, `join()`), it will throw an `InterruptedException`.
- `InterruptedException`
 - A checked exception thrown when a thread is interrupted while it's in a blocking state
 - Catching this exception is often a signal to gracefully stop or exit the current thread
- `Runnable` and `InterruptedException`
 - The `Runnable` interface `run()` method does not declare `InterruptedException` in its signature.
 - This means that if you use a `Runnable` (or a lambda implementing `Runnable`), you cannot propagate `InterruptedException`.
 - Instead, you must handle it inside the `run()` method.

Thread Safety

- Threads primarily communicate by sharing access to common resources
- This form of communication is efficient and fast, but also error-prone without proper care
- Multi-threaded communication can lead to various issues collectively known as **race conditions**, including:
 - thread interference
 - memory consistency errors
 - ...

Thread Interference

- Consider a situation where two threads are operating on the same object at the same time.
- Interference happens when two operations, running in different threads, but acting on the same data, *interleave*. This means that the two operations consist of multiple steps, and the sequences of steps overlap.

```
class Counter {  
    private int c = 0;  
  
    public void increment() {  
        c++;  
    }  
  
    public void decrement() {  
        c--;  
    }  
  
    public int value() {  
        return this.c;  
    }  
}
```

Thread Interference

- Virtual Machines like JVM will decompose the c++ statement into following steps
 - Retrieve the current value of c
 - Increment the retrieved value by 1
 - Store the incremented value back in c
 - “c++ is not atomic”
- What if thread A is calling increment and thread B is calling decrement? (What will be the result?)
 - Code example

```
class Counter {  
    private int c = 0;  
  
    public void increment() {  
        c++;  
    }  
  
    public void decrement() {  
        c--;  
    }  
  
    public int value() {  
        return this.c;  
    }  
}
```

Memory Consistency Errors

- A memory consistency error occurs when different threads have inconsistent views of shared data.
- Why it happens?
 - Modern CPUs and the JVM may cache values or reorder instructions for performance.
 - If thread 1 changes some data, other threads may still see an expired, cached copy of the data.
- **volatile keyword**
 - Declaring a variable as volatile ensures that:
 - All reads go directly to main memory (not a thread's cache).
 - All writes are immediately flushed to main memory.

Synchronization

- The tool needed to prevent these thread safety issues is **synchronization**.
- In Java, synchronization can be achieved in two ways:
 - Synchronized method
 - Synchronized block

```
public synchronized void increment() {  
    c++; // this.c++;  
}  
public void decrement() {  
    synchronized(this) {  
        c--; // this.c--;  
    }  
}
```

Synchronized Keyword

- When one thread is executing a synchronized method on an object, other threads attempting to invoke any synchronized method on the same object are blocked until the first thread finishes.
- Upon exiting a synchronized method, a happens-before relationship is established with any subsequent synchronized method calls on the same object. This ensures that all changes made to the object's state are visible to other threads.
- Think about synchronization in terms of locks — each object has an intrinsic lock, and synchronized methods require acquiring that lock before proceeding.

Synchronized Block

- Synchronized block can be used to perform synchronization on any specific resource of a part of the method
- A synchronized method is essentially a special case of a synchronized block that locks on **this** for the entire method

```
class SynchronizedCounter {  
    private int c = 0;  
  
    public synchronized void increment() {  
        c++;  
    }  
  
    public synchronized void decrement() {  
        c--;  
    }  
  
    public synchronized int value() {  
        return c;  
    }  
}
```

Demon Threads

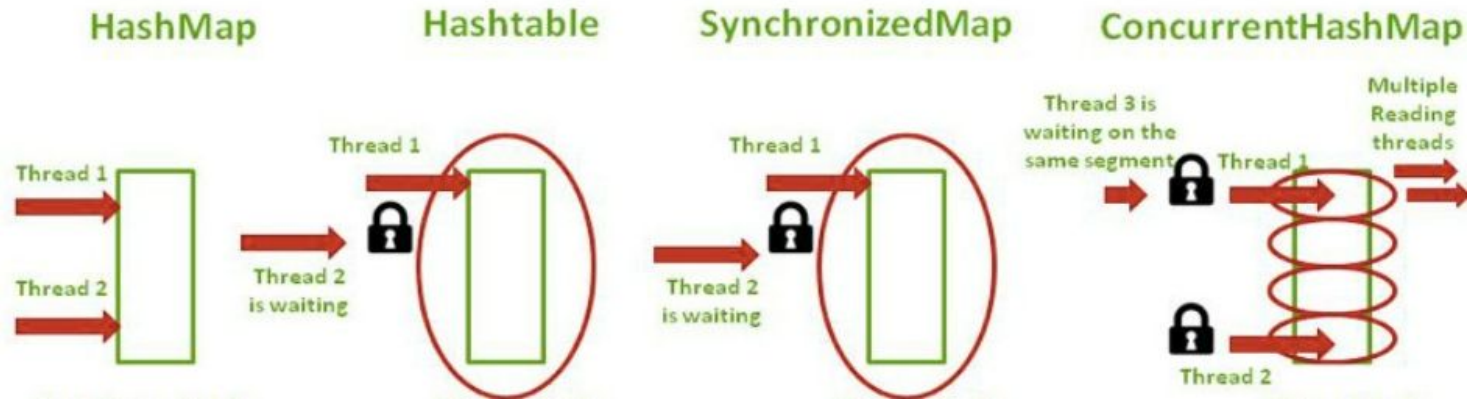
- A low priority thread provides services to user threads for background supporting tasks
- Its life depend on the mercy of user thread
 - i.e. when the user thread dies, JVM terminates this thread immediately, regardless of if its daemon threads are still executing or not
- How to create
 - `public void setDaemon(boolean status)` – mark the current thread as daemon thread or user thread
 - `public boolean isDaemon()` – check that whether the thread is daemon thread
- For example:
 - Some of the MMORPG games will have daemon threads to load the maps, while the user threads work on the more important things like character action. It makes no sense to continue loading the previous map after you quit the game.

java.util.concurrent

- Java.util.concurrent
 - Utility class, includes some useful interfaces in concurrent programming
 - ConcurrentHashMap
 - ReentrantLock
 - AtomicInteger
 - BlockingQueue

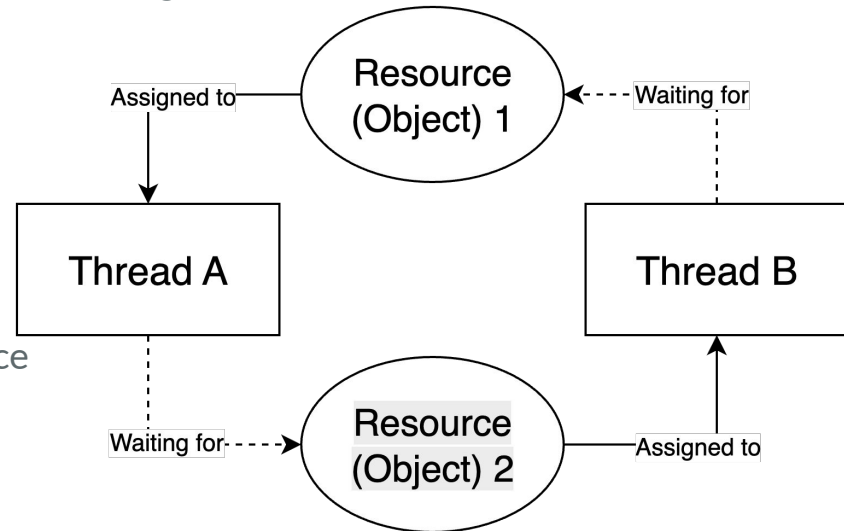
HashMap Thread Safety

- Is HashMap thread safe in Java?
 - What does it mean by being “thread safe”?
 - For example, ...
- Other implementations of hashmap in Java
 - Hashtable (legacy class from Java 1.0)
 - SynchronizedMap (integrated in Collections utility class)
 - ConcurrentHashMap (bucket-level lock, better performance)



Deadlock

- *Deadlock* describes a situation where two or more threads are blocked forever, waiting for each other to release the lock.
- Necessary Conditions
 - Mutual Exclusion - only 1 process can use a resource at a given time
 - Hold and Wait
 - No Preemption
 - Circular Wait
- Solutions
 - Try lock with timeout & retry
 - Lock Ordering
 - Minimize number of locks a thread needs at once
 - ...



Singleton Design Pattern

- Singleton is a design pattern which ensures that one instance of the singleton class exist at a time.
- Two ways to create a singleton class:
 - Eager initialization – The instance is initialized at the time of creation.
 - Lazy initialization – The instance is initialized when **the first time** getInstance() method is called.
- What could be the problem in multithreading environment?
 - How to ensure the thread safety? 2 approaches:
 - **Double check locking** for lazily initialized Singleton classes
 - Synchronized getInstance() method
- Why do we need lazy initialization?

A large red square with a white border, centered on a white background. The text "Thank You!" is written in white inside the square.

Thank You!